

AI OPERATION- COMPLETE PROJECT FLOW DOCUMENTATION

PROJECT OVERVIEW AND PURPOSE

This is an AI-powered facility management system designed to automate issue reporting, problem solver assignment, automated calling, and lifecycle tracking with intelligent escalation mechanisms. The system enables supervisors to report maintenance issues through natural language chat with image uploads, automatically matches problems to available solvers based on location and skill, initiates automated voice calls via Twilio with retry logic, tracks the entire lifecycle from issue creation through resolution, and includes intelligent escalation mechanisms that automatically alert managers and supervisors when issues approach deadlines or when solvers fail to respond within defined timeframes.

COMPLETE SYSTEM WORKFLOW - SEVEN INTERCONNECTED STAGES

Stage One - Issue Creation and AI Extraction

When a supervisor sends a message like "pipe broken in vepery site need to fix in 5 days" along with an image, the AI agent powered by Groq API using llama-3.3-70b-versatile model extracts structured information from the natural language message. The extraction identifies `problem_type` as one of plumbing, electrical, hvac, carpentry, painting, network, or mechanical, `location` as the extracted site name, `days_to_fix` as a number, `priority` as low, medium, or high, a short title with maximum 50 characters, and a detailed description. The AI response is parsed as JSON with temperature set to 0 for consistent deterministic results.

This extracted information is simultaneously saved to three database tables. The `issues` table stores the main problem details with `site_id` referencing the site location, `raised_by_supervisor_id` referencing the supervisor who created the issue, `title` and `description` from AI extraction, `priority` determined from urgency keywords, `deadline_at` timestamp calculated by adding the specified days to the current timestamp, `status` defaulting to OPEN, `track_status` set to awaiting_solver, `latitude` and `longitude` coordinates for location tracking, and timestamps for `created_at` and `updated_at`. The `issue_images` table stores the uploaded image URL with `issue_id` linking back to the issue, `uploaded_by_user_id` referencing the supervisor, `image_url` storing the ImageKit CDN URL, `image_type` marked as BEFORE to indicate it's the initial problem photo, `ai_flag` defaulting to NOT_CHECKED, and timestamps. The `chat_history` table logs the user's message with `user_id` referencing the supervisor, `issue_id` linking to the created issue, `role_in_chat` set to USER, `message` containing the original text, `attachments` as JSONB array containing the image URL, and `created_at` timestamp.

Stage Two - Smart Solver Matching Algorithm

Once the issue is created, the system performs intelligent solver matching by querying the `problem_solver_skills` table. The matching algorithm searches for solvers who have the required `skill_type` matching the extracted `problem_type`, and who are located at the specified `site_id` or have `site_id` as NULL indicating they work across all sites. The algorithm considers priority scores where higher priority values indicate preferred solvers, checks `is_available` flag to ensure the solver is currently active and not on leave, and evaluates current workload by counting active assignments for each potential solver.

The matching query first looks for solvers with exact site matches and highest priority scores, then falls back to solvers who work across all sites, and finally selects the solver with the lowest current workload among qualified candidates. When a match is found, a new record is created in the `issue_assignments` table with `issue_id` linking to the problem, `assigned_to_solver_id` identifying the selected problem solver, `assigned_by_supervisor_id` set to the supervisor or system user, `due_date` copied from the issues `deadline_at` timestamp, `status` set to `ACTIVE` indicating current assignment, and timestamps. Simultaneously, the `issues` table is updated setting `status` to `ASSIGNED` and `track_status` to `awaiting_solver`.

Stage Three - Automated Calling with Retry Logic

After solver assignment, the system initiates the automated calling process through Twilio REST API. The first call is made immediately using the `calls.create` method with the `to` parameter set to the solver's phone number from the `users` table, `from` parameter set to the configured Twilio phone number, and `twiml` parameter containing the voice message. The TwiML uses the `Say` verb with `voice` set to `Polly.Aditi` for Indian English and message content stating Hello followed by the solver name, you have a new work assignment, the problem description from the issue, the location site name, the deadline timestamp formatted in readable format, and asking the solver to confirm their availability.

After initiating the call, a record is created in the `call_logs` table with `assignment_id` linking to the issue assignment, `solver_id` identifying who was called, `attempt_number` set to 1 for the first call, `initiated_at` timestamp recording when the call was made, `answered_at` and `ended_at` both NULL initially, and `status` set to `INITIATED`. The system then monitors the call status by polling the Twilio API using the `call_sid` to check the `status` property every 2 seconds for up to 20 seconds.

If the call status becomes in-progress or completed, the system updates the `call_logs` record setting `status` to `ANSWERED`, `answered_at` to the timestamp when the solver picked up, `ended_at` when the call completes, and `updated_at` timestamp. However, if after 20 seconds the call is not answered and Twilio status is `busy`, `failed`, `no-answer`, or `canceled`, the system updates the `call_logs` record setting `status` to `MISSED` and immediately initiates the next call attempt.

we have to use a celery method to make a repeated call for every two hour total with new call attempts made. Each attempt is logged as a separate record in call_logs with incrementing attempt_number values. The system continues retrying until either the solver answers the call, or the escalation conditions are triggered, or the maximum retry window of 90 seconds is exhausted.

Stage Four - Escalation Checking During Retries

After each missed call attempt, the system performs real-time escalation checks. The system queries the escalation_rules table filtering by the current issue priority to retrieve max_call_attempts and max_time_without_response interval values. For the current assignment, the system counts total missed calls by querying call_logs where assignment_id matches and status equals MISSED, and calculates time elapsed since the first call by finding the minimum initiated_at timestamp and subtracting from current timestamp.

If either condition is met where total missed calls equals or exceeds max_call_attempts, or time elapsed exceeds max_time_without_response interval, the system triggers an escalation event. A new record is created in the escalations table with issue_id linking to the problem, assignment_id linking to the current assignment, escalation_type set to NO_RESPONSE indicating the solver did not answer calls, escalated_to_role copied from the escalation_rules table indicating whether to notify manager, supervisor, or admin, reason as text describing the number of missed calls and time elapsed, total_missed_calls set to the actual count, time_elapsed_without_response set to the calculated interval, notification_sent initially false, and created_at timestamp.

The system then queries the users table to find all users with role matching the escalated_to_role value. For each matching user, an automatic email notification is sent using SMTP containing urgent alert header, issue title and description, site location and address, assigned solver name and phone number, number of missed call attempts, time elapsed without response, priority level highlighted, and deadline urgency. After sending all emails, the escalations record is updated setting notification_sent to true and notification_sent_at to current timestamp. The issues table is also updated setting status to ESCALATED. A system message is inserted into chat_history with role_in_chat set to SYSTEM, issue_id linking to the escalated issue, and message describing the escalation event.

Stage Five - Solver Work Completion and AI Verification

When the solver answers the call and accepts the assignment, they can update the issue status through the mobile app or chatbot. When the solver begins work, the issues table is updated setting status to IN_PROGRESS and track_status to in_progress. Once the solver completes the repair work, they upload an after photo which creates a new record in the issue_images table with issue_id linking to the problem, uploaded_by_user_id referencing the solver, image_url storing the

ImageKit CDN URL for the completion photo, image_type marked as AFTER to distinguish from the initial BEFORE photo, ai_flag initially set to NOT_CHECKED, and timestamps.

The system can optionally trigger AI vision verification on the after photo to check if the repair looks complete. The verification function analyzes the image using vision AI models and returns structured results. If the AI confidence score is above 0.85, the ai_flag is set to OK indicating the repair appears satisfactory. If confidence is below 0.85, the ai_flag is set to SUSPECT indicating potential issues. The ai_details JSONB field stores detailed analysis including confidence score and problem-specific findings like leak_detected false for plumbing or wiring_fixed true for electrical work.

After uploading the after photo, the issues table is updated setting status to RESOLVED_PENDING_REVIEW and track_status to awaiting_review. The issue_assignments table is updated setting status to ACTIVE but indicating work completion pending supervisor approval. A notification is sent to the original supervisor who raised the issue informing them that the work is complete and ready for review.

Stage Six - Complaint Handling and Reopening

When the supervisor inspects the completed work, if they find it unsatisfactory they can raise a complaint through the chatbot by sending a message like "work not done properly, leak still visible" along with an optional evidence photo. The AI agent detects the complaint intent and processes it accordingly. A new record is created in the complaints table with issue_id linking to the original problem, assignment_id linking to the current assignment, raised_by_supervisor_id identifying who filed the complaint, target_solver_id identifying the solver whose work is being contested, complaint_details containing the textual description, complaint_image_url storing the evidence photo if provided, and timestamps.

When a complaint is raised, multiple updates cascade through the system. The issues table is updated setting status to REOPENED and track_status back to in_progress. The issue_assignments table is updated setting status to REOPENED rather than creating a new assignment record, maintaining continuity of the assignment_id. A new round of automated calls is initiated to the same solver using the reopened assignment_id so all call logs reference the same assignment. A system message is logged in chat_history documenting that assignment was reopened after complaint and re-calling the solver. The supervisor who filed the complaint receives confirmation that the issue has been reopened and the solver will be notified.

Stage Seven - Deadline Escalation Background Job

The system runs a scheduled background job every hour that checks for issues approaching their deadlines. The job queries the issues table with conditions filtering

where status is not COMPLETED and not CANCELLED, and calculates the time difference between deadline_at and current timestamp. Issues with less than 24 hours remaining until deadline are flagged for potential escalation.

For each flagged issue, the system first checks if an escalation record already exists with escalation_type DEADLINE_APPROACHING and resolved false. If no such escalation exists, the system proceeds with deadline escalation. The system evaluates if priority should be elevated - if current priority is low or medium, the issues table is updated setting priority to high to ensure appropriate attention.

A new record is created in the escalations table with issue_id linking to the problem, assignment_id if an active assignment exists, escalation_type set to DEADLINE_APPROACHING, escalated_to_role set to manager, reason containing descriptive text like "Issue deadline is in 18 hours and status is still IN_PROGRESS", notification_sent initially false, and timestamps.

The system prepares and sends alert emails to multiple recipients by querying the users table for all users with role manager and all users with role supervisor who manage the same site as the escalated issue. The email subject line states URGENT DEADLINE ALERT followed by the issue title and site location. The email body formatted in HTML contains urgent warning banner, issue title and detailed description, site location and full address, current status and priority level, deadline timestamp with hours remaining highlighted in red, assigned solver name and contact if available, supervisor who originally raised the issue, and call to action stating immediate attention is required. After sending all emails, the escalations record is updated setting notification_sent to true and notification_sent_at to current timestamp. A system message is logged in chat_history documenting the deadline escalation event.

DATABASE SCHEMA - COMPLETE TABLE DEFINITIONS

Table 1 - users

Stores all system users including supervisors, managers, problem solvers, and administrators with authentication and contact information. Columns include id as SERIAL primary key, name as VARCHAR 100 NOT NULL, phone as VARCHAR 20 UNIQUE NOT NULL for authentication and calling, email as VARCHAR 100 for notifications, role as ENUM with values supervisor, problemsolver, manager NOT NULL with check constraint, is_active as BOOLEAN DEFAULT TRUE to track account status, created_at as TIMESTAMPTZ DEFAULT NOW, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_users_role on role column for role-based filtering, idx_users_phone on phone column for login lookups, and idx_users_email on email column for email notifications.

Table 2 - sites

Stores all facility locations with GPS coordinates for matching and tracking. Columns include id as SERIAL primary key, name as VARCHAR 100 NOT NULL for site identification, location as VARCHAR 200 for human-readable address, latitude as DECIMAL 10 comma 8 for precise GPS coordinates, longitude as DECIMAL 11 comma 8 for precise GPS coordinates, created_at as TIMESTAMPTZ DEFAULT NOW, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_sites_location on location column for fast location-based queries.

Table 3 - supervisor_sites

Junction table establishing many-to-many relationship between supervisors and sites since one supervisor can manage multiple sites. Columns include supervisor_id as INT foreign key referencing users id, and site_id as INT foreign key referencing sites id. No timestamps needed as mentioned in requirements. Composite primary key on both columns to prevent duplicate assignments.

Table 4 - issues

Main table storing all reported maintenance issues with full status tracking and priority management. Columns include id as SERIAL primary key, site_id as INT foreign key referencing sites id NOT NULL linking issue to location, raised_by_supervisor_id as INT foreign key referencing users id NOT NULL tracking who reported the issue, title as VARCHAR 200 NOT NULL for short problem description, description as TEXT NOT NULL for detailed problem explanation, priority as ENUM with values low, medium, high DEFAULT medium, deadline_at as TIMESTAMPTZ for calculated deadline timestamp, status as ENUM with values OPEN, ASSIGNED, IN_PROGRESS, RESOLVED_PENDING_REVIEW, COMPLETED, REOPENED, ESCALATED DEFAULT OPEN tracking lifecycle, track_status as VARCHAR 100 for additional tracking like awaiting_solver or awaiting_review, latitude as DECIMAL 10 comma 8 for issue location, longitude as DECIMAL 11 comma 8 for issue location, created_at as TIMESTAMPTZ DEFAULT NOW, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_issues_status on status column for filtering, idx_issues_priority on priority column for priority-based queries, idx_issues_site on site_id for location filtering, idx_issues_supervisor on raised_by_supervisor_id for supervisor dashboards, and idx_issues_deadline on deadline_at for deadline checks.

Table 5 - issue_assignments

Tracks which solver is assigned to which issue with due dates and assignment status, allowing multiple assignments per issue if reopened. Columns include id as SERIAL primary key, issue_id as INT foreign key referencing issues id NOT NULL, assigned_to_solver_id as INT foreign key referencing users id NOT NULL identifying the problem solver, assigned_by_supervisor_id as INT foreign key referencing users id NOT NULL tracking who made the assignment, due_date as

TIMESTAMPTZ NOT NULL copied from issues deadline_at, status as ENUM with values ACTIVE, COMPLETED, REOPENED DEFAULT ACTIVE, created_at as TIMESTAMPTZ DEFAULT NOW, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_assignments_issue on issue_id for finding all assignments for an issue, idx_assignments_solver on assigned_to_solver_id for solver workload tracking, idx_assignments_status on status column for active assignment filtering, and idx_assignments_due_date on due_date for overdue checks.

Table 6 - call_logs

Logs every call attempt made to problem solvers through Twilio including retry attempts and call outcomes for escalation checking. Columns include id as SERIAL primary key, assignment_id as INT foreign key referencing issue_assignments id linking calls to assignments, solver_id as INT foreign key referencing users id identifying who was called, attempt_number as INT DEFAULT 1 tracking retry sequence, initiated_at as TIMESTAMPTZ NOT NULL when call was made, answered_at as TIMESTAMPTZ NULLABLE when solver picked up, ended_at as TIMESTAMPTZ NULLABLE when call terminated, status as ENUM with values INITIATED, MISSED, ANSWERED NOT NULL showing call outcome, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_call_logs_assignment on assignment_id for counting attempts, idx_call_logs_solver on solver_id for tracking responsiveness, idx_call_logs_status on status column for missed call queries, idx_call_logs_initiated on initiated_at for time-based calculations, and idx_call_logs_attempt on attempt_number for retry tracking.

Table 7 - issue_images

Stores all images related to issues with separate rows for BEFORE and AFTER photos, not the same row. Columns include id as SERIAL primary key, issue_id as INT foreign key referencing issues id linking images to issues, uploaded_by_user_id as INT foreign key referencing users id tracking who uploaded either supervisor for BEFORE or solver for AFTER, image_url as VARCHAR 500 NOT NULL storing ImageKit CDN URL, image_type as ENUM with values BEFORE, AFTER DEFAULT AFTER distinguishing initial problem photo from completion photo, ai_flag as ENUM with values OK, SUSPECT, NOT_CHECKED DEFAULT NOT_CHECKED indicating AI verification result, ai_details as JSONB for storing AI analysis like confidence scores and specific findings, created_at as TIMESTAMPTZ DEFAULT NOW, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_issue_images_issue on issue_id for finding all images for an issue, idx_issue_images_type on image_type for separating before and after, idx_issue_images_flag on ai_flag for finding suspect repairs, and idx_issue_images_user on uploaded_by_user_id for tracking uploads.

Table 8 - complaints

Handles complaints raised by supervisors when work is incomplete or unsatisfactory, triggering reopening of assignments. Columns include `id` as SERIAL primary key, `issue_id` as INT foreign key referencing issues `id` NOT NULL, `assignment_id` as INT foreign key referencing issue_assignments `id` NOT NULL, `raised_by_supervisor_id` as INT foreign key referencing users `id` NOT NULL, `target_solver_id` as INT foreign key referencing users `id` NOT NULL identifying whose work is contested, `complaint_details` as TEXT NOT NULL describing the problem with completed work, `complaint_image_url` as VARCHAR 500 NULLABLE for optional evidence photo, `created_at` as TIMESTAMPTZ DEFAULT NOW, and `updated_at` as TIMESTAMPTZ DEFAULT NOW. Required indexes include `idx_complaints_issue` on `issue_id` for finding complaints per issue, `idx_complaints_assignment` on `assignment_id` for assignment history, and `idx_complaints_solver` on `target_solver_id` for tracking solver performance.

Table 9 - problem_solver_skills

Maps solvers to their skills and preferred sites with priority scoring for intelligent matching. Columns include `id` as SERIAL primary key, `solver_id` as INT foreign key referencing users `id` NOT NULL, `skill_type` as VARCHAR 50 NOT NULL with values like plumber, electrician, hvac, carpentry, painting, network, or mechanical, `site_id` as INT foreign key referencing sites `id` NULLABLE where NULL means works at all sites, `priority` as INT DEFAULT 1 with range 1 to 10 where higher means preferred solver, `is_available` as BOOLEAN DEFAULT TRUE for tracking availability, `created_at` as TIMESTAMPTZ DEFAULT NOW, and `updated_at` as TIMESTAMPTZ DEFAULT NOW. Required indexes include `idx_solver_skills_skill` on `skill_type` for matching queries, `idx_solver_skills_site` on `site_id` for location-based matching, `idx_solver_skills_solver` on `solver_id` for solver profiles, and composite index `idx_solver_skills_skill_site` on both `skill_type` and `site_id` for optimized matching.

Table 10 - escalation_rules

Defines escalation criteria for different priority levels including maximum call attempts and time thresholds before triggering alerts. Columns include `id` as SERIAL primary key, `priority` as ENUM with values low, medium, high NOT NULL, `max_call_attempts` as INT NOT NULL defining how many missed calls trigger escalation, `max_time_without_response` as INTERVAL NOT NULL defining time duration like '02:00:00' for 2 hours, `escalate_to_role` as VARCHAR 50 NOT NULL with values manager, supervisor, or admin defining which role receives alerts, `created_at` as TIMESTAMPTZ DEFAULT NOW, and `updated_at` as TIMESTAMPTZ DEFAULT NOW. Required indexes include `idx_escalation_rules_priority` on `priority` column for fast lookup during checks. Sample rules show high priority escalates to manager after 3 missed calls or 2 minutes, medium priority escalates to supervisor after 2 missed calls or 1 hour, and low priority escalates to manager after 5 missed calls or 4 hours.

Table 11 - escalations

Logs all escalation events including both call-based escalations when solvers don't respond and deadline-based escalations when issues approach due dates. Columns include id as SERIAL primary key, issue_id as INT foreign key referencing issues id NOT NULL, assignment_id as INT foreign key referencing issue_assignments id NULLABLE for call escalations, escalation_type as VARCHAR 30 NOT NULL with values NO_RESPONSE for missed calls, DEADLINE_APPROACHING for deadline warnings, or MANUAL for supervisor-triggered escalations, escalated_to_role as VARCHAR 20 NOT NULL indicating which role was notified, escalated_by_user_id as INT foreign key referencing users id NULLABLE for manual escalations, reason as TEXT NOT NULL describing why escalation occurred, total_missed_calls as INT for call escalations, time_elapsed_without_response as INTERVAL for call escalations, notification_sent as BOOLEAN DEFAULT FALSE tracking email status, notification_sent_at as TIMESTAMPTZ NULLABLE, resolved as BOOLEAN DEFAULT FALSE tracking if escalation was addressed, resolved_at as TIMESTAMPTZ NULLABLE, created_at as TIMESTAMPTZ DEFAULT NOW, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_escalations_issue on issue_id for issue history, idx_escalations_type on escalation_type for filtering, idx_escalations_resolved on resolved column for active escalations, and idx_escalations_created on created_at for time-based queries.

Table 12 - chat_history

Stores all chat messages between users and AI including system notifications and escalation alerts for complete communication tracking and audit trail. Columns include id as SERIAL primary key, user_id as INT foreign key referencing users id NULLABLE where NULL means system or AI message, issue_id as INT foreign key referencing issues id NULLABLE for issue-related messages, role_in_chat as ENUM with values USER, AI, SYSTEM indicating message source, message as TEXT NOT NULL containing message content, attachments as JSONB DEFAULT empty array for storing file URLs and metadata, created_at as TIMESTAMPTZ DEFAULT NOW, and updated_at as TIMESTAMPTZ DEFAULT NOW. Required indexes include idx_chat_issue on issue_id for retrieving issue conversation history, idx_chat_user on user_id for user message history, and idx_chat_created on created_at with descending order for recent message retrieval.